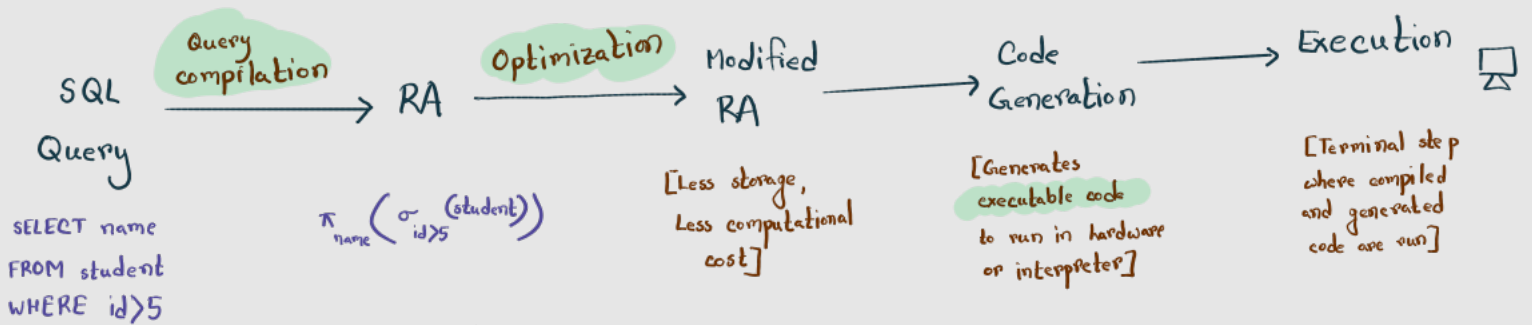
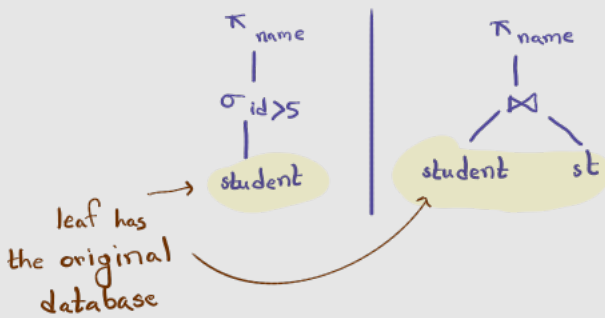


Query Processing

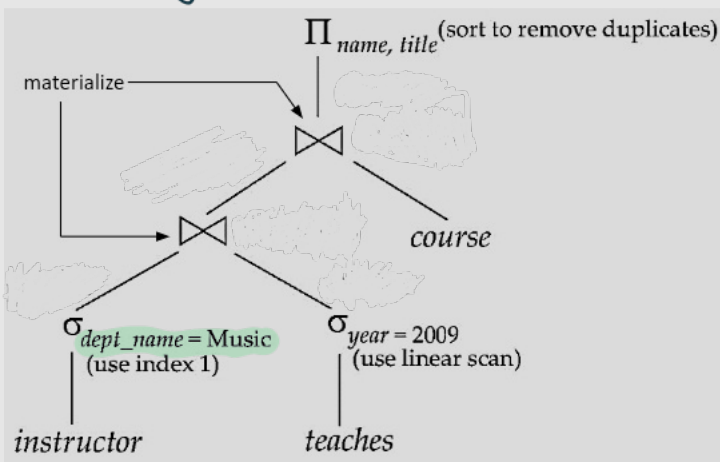
* 'Why use X instead of Y' - Better optimization



RA tree: Handled by backend. It is made in a **recursive** way which makes complex queries simpler.



Pipelining



* Tuples (Building blocks) go up from leaf to the parent

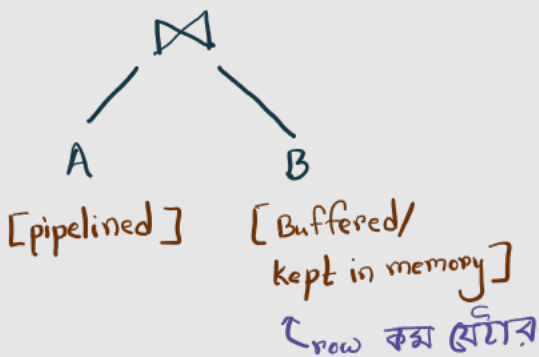
one by one

* Takes less memory

C
M
M

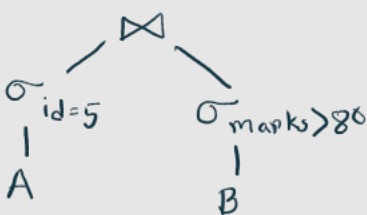
C
M
M

Limitation / Materialization: When Joining



* If $\sigma_{id=5}(A)$ and $\sigma_{marks > 80}(B)$ joined, which one to store?

Ans:



store A. Equal (=) sign স্টোরে row কত হয় Pipeline B.

Alternative algorithm:

selected based on : (i) Nature of indexing
(ii) Overall best fit

1. External sorting
2. Selection
3. Join
4. Projection
5. Aggregate operation

① External sorting : Sorting based on the attribute which wasn't previously used for sorting.

↓

id	name	position
1		3
2		4
3		2
4		7
5		5
6		6
7		11
8		13
9		12

↓

Block/chunck/page sort



* Partition based on chunck/block/page

* $O(\log n)$

* Use merge sort

* Reduce I/O pass

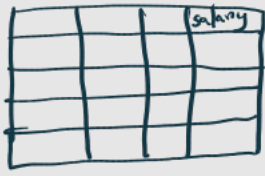
② Selection :

(a) One condition:

SELECT WHERE salary > 50K

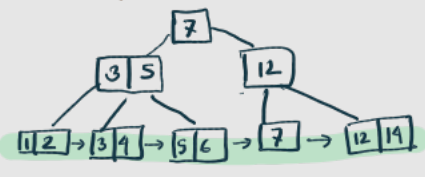
Works for : <, <=, >, >= etc.

Linear scan
[Brute Force]



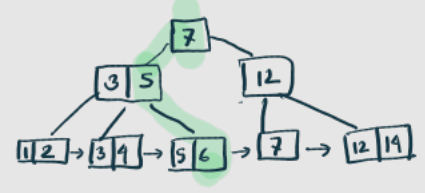
$O(n) \rightarrow$ slow

Index scan
[Indexing/B+ tree আছে]



Read leaf chain
 $O(k)$

Index seek
[Indexing/B+ tree আছে]

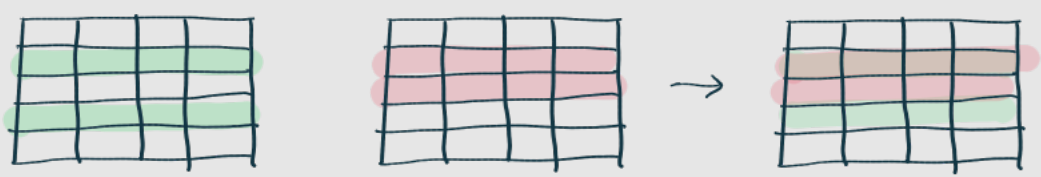


Binary search
 $O(\log n)$

(b) Two condition/ Disjunctive [OR] :

- * Use pipeline
- * Use chain rule

.... WHERE salary > 50K OR uni = 'BRACU'



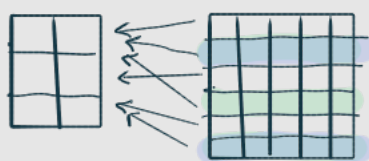
(b) Two condition/ Conjunctive [AND] :

- * No pipeline

① Single search index

.... WHERE salary > 50K AND uni = 'BRACU'

Indexing করা



AND
✓ ✗

Filter using the indexed table

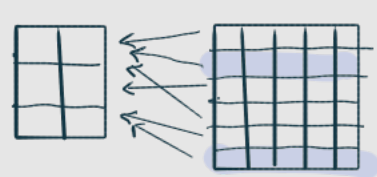
Then compare with non indexed one

② Composite indexing :

.... WHERE salary > 50K AND uni = 'BRACU'

Indexing করা

Indexing করা

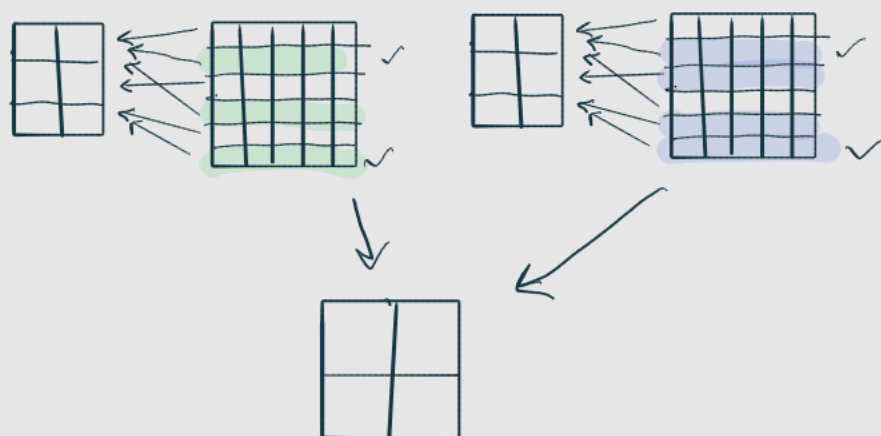


AND
✓ ✓

iii) Intersecting Record Point:

জানাদা

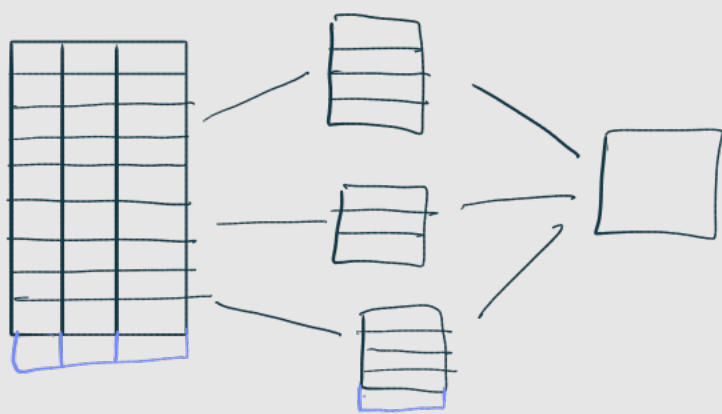
.... WHERE salary > 50K AND uni = 'BRACU'
 Indexing করা Indexing করা



SELECT salary FROM employee WHERE salary > 50K AND shift = 'Day';

$\sigma_{\text{shift='Day'}} \left(\sigma_{s>50K} (E) \right)$
 Independent

③ Aggregate Function:

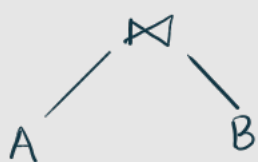


split — Apply — Combine

- AVG()
- MIN()
- MAX()
- SUM()
- COUNT()

* If new tuple appears it updates a running group

④ Projection: Do projection as close to the leaf of query tree as possible



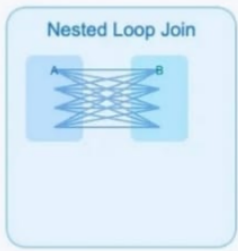
❌ Join → Projection

✅ Projection → Join

Benefit: → Reduce overhead

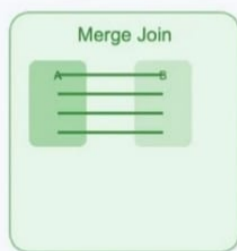
→ Unnecessary attribute/tuple removed

④ Join :



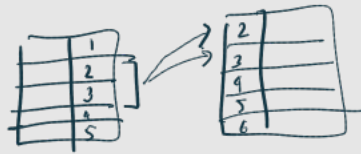
Nested Loop Join

- * Every single row loops through the entire 2nd table.
- * $O(n \times m)$
- * Time complexity [High]
- * Space complexity [Low]

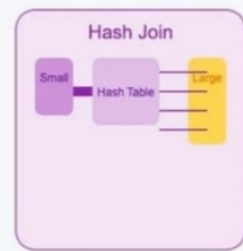


Merge Join

- * Separately sorted

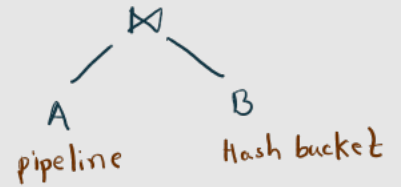


- * Time complexity [Mid]
- * Space complexity [High]



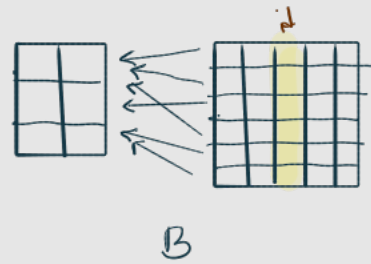
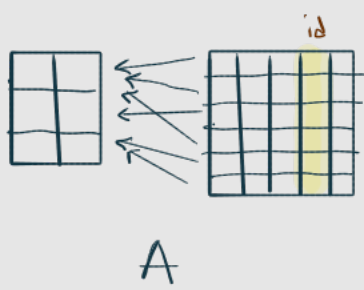
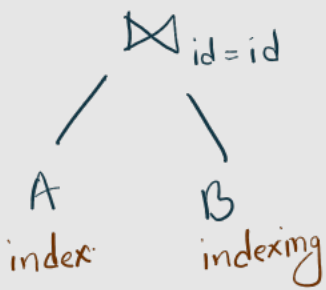
Hash Join

- * Time complexity [Low]
- * Space complexity [Medium]



When can we use index based join in both table bypassing traditional joins?

Ans: Both are indexed based on the column being joined



Sample:

Consider a database containing two tables:
Student(ID, Name, DeptName) and
Enrollment(StudentID, CourseID, Grade). A
user executes the following SQL query:

```
SELECT Student.Name, Enrollment.CourseID
FROM Student JOIN Enrollment ON
Student.ID = Enrollment.StudentID
WHERE Student.DeptName = 'CSE';
```

- Identify the primary advantage of converting this SQL query into a Relational Algebra Query Tree during compilation.
- Construct an optimized sequence of operations using a basic heuristic rule (pushing selection down).
- State why filtering the Student table early reduces execution cost.

(a) Helps DB to optimize operation without changing the final result

(b) Projection $\rightarrow$ JOIN $\rightarrow$ Select

(c) Projection; removes non-CSE. Reduce overhead

